

Clase 9 – Advanced React

IIC2182 – Interfaces y Experiencia de Usuario



PONTIFICIA
UNIVERSIDAD
CATÓLICA
DE CHILE

Primer Semestre 2026

Agenda

Hoy profundizamos en patrones y herramientas que hacen React escalable en proyectos reales.

Optimización y Patrones

 `useEffect Cleanup` — evitar efectos secundarios no deseados

 `useCallback` — estabilizar referencias de funciones

 Custom Hooks — extraer lógica reutilizable

 Composición — construir UIs flexibles

Librería Esencial

 Zustand — estado global simple y escalable

useEffect Cleanup

La función de retorno que evita memory leaks

¿Qué es la cleanup function?

“ Definición

"Cuando un `useEffect` retorna una función, React la ejecuta antes de re-ejecutar el efecto y cuando el componente se desmonta. Sirve para limpiar recursos."

```
useEffect(() => {  
  // 1. Setup: se ejecuta cuando el componente se monta o las deps cambian  
  const timer = setInterval(() => console.log('tick'), 1000)  
  
  // 2. Cleanup: se ejecuta ANTES del próximo setup y al desmontar  
  return () => {  
    clearInterval(timer)  
  }  
}, [dependency])
```



Al montar

Se ejecuta el setup (el cuerpo del effect)



Al cambiar deps

Primero cleanup del effect anterior, luego setup nuevo



Al desmontar

Se ejecuta cleanup para liberar recursos

¿Por qué importa? — Memory Leaks

“ Definición

"Un memory leak en useEffect ocurre cuando un efecto secundario (timer, suscripción, request) sigue ejecutándose después de que el componente se desmontó."

Sin cleanup, los efectos secundarios se acumulan cada vez que el componente se re-renderiza

⚠️ ❌ Sin cleanup — memory leak

```
useEffect(() => {  
  // Se crea un nuevo interval en CADA render  
  // Los anteriores siguen corriendo ✨  
  setInterval(() => {  
    setCount(c => c + 1)  
  }, 1000)  
}, [])
```

✅ ✅ Con cleanup — limpio

```
useEffect(() => {  
  const id = setInterval(() => {  
    setCount(c => c + 1)  
  }, 1000)  
  
  // Limpia el interval anterior  
  return () => clearInterval(id)  
}, [])
```

Casos comunes de cleanup

Timers

```
useEffect(() => {  
  const id = setTimeout(() => doSomething(), 1000)  
  return () => clearTimeout(id)  
}, [dep])
```

Event Listeners

```
useEffect(() => {  
  const handler = (e) => setPos(e.clientY)  
  window.addEventListener('scroll', handler)  
  return () => window.removeEventListener('scroll', h  
}, [])
```

Fetch con AbortController

```
useEffect(() => {  
  const ctrl = new AbortController()  
  fetch(url, { signal: ctrl.signal })  
    .then(r => r.json()).then(setData)  
  return () => ctrl.abort()  
}, [url])
```

WebSocket / Subscripciones

```
useEffect(() => {  
  const ws = new WebSocket(url)  
  ws.onmessage = (e) => setMsg(e.data)  
  return () => ws.close()  
}, [url])
```

Optimización de Re-renders

Entender el problema y las herramientas para resolverlo

¿Por qué importan los re-renders excesivos?

“ Definición

"Un re-render ocurre cuando React re-ejecuta un componente para calcular su nuevo JSX. El componente se re-renderiza cuando cambia su estado, sus props, o el contexto que consume."

El efecto cascada

Cuando un componente se re-renderiza, **todos sus hijos también** se re-renderizan por defecto

Un cambio en el root puede causar que **cientos de componentes** se re-ejecuten

El resultado: UI lenta, input laggy, animaciones entrecortadas



React.memo

Evita re-renders si las props no cambiaron



useCallback

Estabiliza funciones para que memo funcione



useTransition

Prioriza updates urgentes sobre las pesadas

React.memo — Evitar re-renders innecesarios

“ Definición

"React.memo envuelve un componente para que solo se re-renderice si sus props cambian. Si las props son iguales al render anterior, React reutiliza el resultado previo."

```
// Sin memo: se re-renderiza SIEMPRE que el padre se re-renderiza
function Greetings({ name }: { name: string }) {
  return <div>{name}</div>
}

// Con memo: solo se re-renderiza si 'name' cambió
const MemoizedGreetings = memo(function MemoizedGreetings({ name }: { name: string }) {
  return <div>{name}</div>
})
```

¿Cómo compara props?

Usa **shallow equality** (`===`) para cada prop

Strings, numbers, booleans → compara por valor 

Objetos, arrays, funciones → compara por **referencia**

El problema con funciones

Si el padre pasa una función como prop...

...la función se **recrea en cada render**

...memo ve una referencia **nueva** → re-renderiza igual

Solución: `useCallback`

useCallback al rescate

“ Definición

"useCallback memoriza una función para que mantenga la misma referencia entre renders, a menos que cambien sus dependencias."

```
const memoizedFn = useCallback(  
  (args) => { /* lógica */ },  
  [dependencia1, dependencia2] // solo se recrea si cambian estas  
)
```

✗ Sin useCallback

```
function Parent() {  
  // Se recrea en CADA render  
  const handleDelete = (id: number) => {  
    setItems(prev => prev.filter(  
      item => item.id !== id  
    ))  
  }  
  return <List onDelete={handleDelete} />  
}
```

✓ Con useCallback

```
function Parent() {  
  // Misma referencia entre renders  
  const handleDelete = useCallback(  
    (id: number) => {  
      setItems(prev => prev.filter(  
        item => item.id !== id  
      ))  
    }, [] // sin deps externas  
  )  
  return <List onDelete={handleDelete} />  
}
```

useCallback + memo = la combinación

memo evita re-renders si las props no cambian. useCallback asegura que las funciones pasadas como props no cambien.

```
// Padre: useCallback mantiene la referencia estable
function TodoList() {
  const [todos, setTodos] = useState<Todo[]>([])

  const handleToggle = useCallback((id: number) => {
    setTodos(prev => prev.map(t => t.id === id ? { ...t, done: !t.done } : t))
  }, []) // [] = misma función siempre

  return todos.map(todo =>
    <TodoItem key={todo.id} todo={todo} onToggle={handleToggle} />
  )
}

// Hijo: memo solo re-renderiza si todo u onToggle cambiaron
const TodoItem = memo(function TodoItem({ todo, onToggle }: Props) {
  return <div onClick={() => onToggle(todo.id)}>{todo.title}</div>
})
```



memo sin useCallback no sirve para props de tipo función. useCallback sin memo no tiene efecto. **Se necesitan mutuamente.**

¿Cuándo usar useCallback?

No todas las funciones necesitan ser memorizadas — úsalo con criterio

✓ Sí usar

- Props a componentes memorizados con `React.memo`
- Dependencias de `useEffect`
- Handlers en listas con muchos ítems
- Funciones en contextos

⚠ No necesario

- Handlers de eventos simples que se pasan a pocos componentes
- Funciones que no se pasan como props
- Componentes que ya son baratos de re-renderizar
- Optimización prematura sin medir

useTransition — Actualizaciones pesadas bloquean la UI

Filtrar 2.000+ ítems en cada tecla congela el input

```
function App() {
  const [value, setValue] = useState('')
  const [filtered, setFiltered] = useState(hugeList) // 2.000 productos

  const handleChange = (newValue: string) => {
    setValue(newValue)
    // ❌ Bloquea el input mientras filtra 2.000 ítems
    setFiltered(hugeList.filter(item =>
      item.name.toLowerCase().includes(newValue.toLowerCase())
    ))
  }

  return (
    <
      <input value={value} onChange={e => handleChange(e.target.value)} />
      <ProductList items={filtered} />
    </>
  )
}
```

useTransition al rescate

“ Definición

"useTransition permite marcar actualizaciones como no urgentes. React prioriza las interacciones del usuario (input) y deja las operaciones pesadas (filtrado) para después."

```
function App() {  
  const [value, setValue] = useState('')  
  const [filtered, setFiltered] = useState([ ...hugeList])  
  const [isPending, startTransition] = useTransition()  
  
  const handleChange = (newValue: string) => {  
    setValue(newValue) // ← urgente: actualiza el input YA  
    startTransition(() => { // ← no urgente: puede esperar  
      setFiltered(hugeList.filter(item =>  
        item.name.toLowerCase().includes(newValue.toLowerCase())  
      ))  
    })  
  })  
}  
  
return (  
  <input value={value} onChange={e => handleChange(e.target.value)} />  
)
```

¿Cuándo usar useTransition?

✓ Sí usar

- Filtrar o ordenar listas grandes (1.000+ ítems)
- Cálculos pesados que ralentizan la UI
- Renders complejos (gráficos, tablas de datos)
- Navegación de tabs con contenido pesado

⚠ No necesario

- Actualizaciones pequeñas y rápidas (toggles, counters)
- Llamadas a APIs (usar loading states)
- Animaciones (usar CSS o librerías de animación)
- 50.000+ elementos DOM (necesitas virtualización)

💡 `useTransition` no hace las cosas más rápidas — hace que las cosas **correctas sean rápidas** (interacciones del usuario) y las pesadas puedan esperar.

React Design Patterns

Patrones que hacen tu código más limpio y reutilizable

Custom Hooks

“ Definición

"Un custom hook es una función que empieza con `use` y encapsula lógica reusable con otros hooks de React."

💡 ¿Por qué?

Reutilización — la misma lógica en múltiples componentes sin duplicar código

Separación — aísla la lógica del estado de la UI que lo consume

Testing — puedes testear la lógica independiente del componente

```
// ❌ Lógica mezclada en el componente
function SearchPage() {
  const [query, setQuery] = useState('')
  const [debouncedQuery, setDebouncedQuery] = useState('')

  useEffect(() => {
    const timer = setTimeout(() => setDebouncedQuery(query), 300)
    return () => clearTimeout(timer)
  }, [query])

  // ... render con query y debouncedQuery
}
```

Custom Hooks — Ejemplos

useDebounce

```
function useDebounce<T>(value: T, delay: number): T {
  const [debounced, setDebounced] = useState(value)

  useEffect(() => {
    const timer = setTimeout(
      () => setDebounced(value),
      delay
    )
    return () => clearTimeout(timer)
  }, [value, delay])

  return debounced
}

// Uso:
const debouncedQuery = useDebounce(query, 300)
```

useLocalStorage

```
function useLocalStorage<T>(
  key: string,
  initial: T
) {
  const [value, setValue] = useState<T>(() => {
    const stored = localStorage.getItem(key)
    return stored ? JSON.parse(stored) : initial
  })

  useEffect(() => {
    localStorage.setItem(key, JSON.stringify(value))
  }, [key, value])

  return [value, setValue] as const
}

// Uso:
const [theme, setTheme] = useLocalStorage('theme', 'light')
```

Composición con Children

“ Definición

"La composición es el patrón fundamental de React: construir componentes complejos combinando componentes simples a través de `children`."

✗ Prop Drilling

```
// Rígido: Layout decide todo
function Layout({
  headerTitle,
  sidebarItems,
  content,
  footerLinks,
}) {
  return (
    <div>
      <Header title={headerTitle} />
      <Sidebar items={sidebarItems} />
      <Main>{content}</Main>
      <Footer links={footerLinks} />
    </div>
  )
}
```

✓ ✓ Composición

```
// Flexible: el padre decide qué va dentro
function Layout({ children }) {
  return (
    <div className="max-w-6xl mx-auto">
      <Header />
      <main>{children}</main>
      <Footer />
    </div>
  )
}

// Uso – cada página decide su contenido
<Layout>
  <Sidebar />
  <Main />
</Layout>
```

Composición — Slots con Named Children

Para layouts con múltiples zonas, usa props como "slots" en vez de un solo `children`

```
// Componente con múltiples "slots"
function DashboardLayout({ sidebar, header, children }: {
  sidebar: React.ReactNode
  header: React.ReactNode
  children: React.ReactNode
}) {
  return (
    <div className="grid grid-cols-[250px_1fr] h-screen">
      <aside className="border-r p-4">{sidebar}</aside>
      <div>
        <header className="border-b p-4">{header}</header>
        <main className="p-6">{children}</main>
      </div>
    </div>
  )
}
```

```
// Uso — el consumidor tiene control total de cada zona
<DashboardLayout
  sidebar={<NavigationMenu />}
  header={<SearchBar />}
>
  <TodoList />
</DashboardLayout>
```

¿Por qué composición?



Flexibilidad

Cada consumidor decide qué contenido poner — sin modificar el componente padre



Evita Prop Drilling

Los datos fluyen directo al componente que los necesita, sin pasar por intermediarios



Reusabilidad

El mismo Layout sirve para páginas completamente distintas



Regla práctica

Si tu componente tiene **más de 3-4 props** que son "contenido" (título, subtítulo, contenido, footer...), probablemente debería usar composición con `children` o slots en su lugar.

Higher-Order Components (HOC)

“ Definición

"Un HOC es una función que recibe un componente y retorna un nuevo componente con props transformadas o funcionalidad adicional."

```
// HOC que transforma props antes de pasarlas al componente
function withDefaultUser<P extends { user: User }>(Component: ComponentType<P>) {
  return function WithDefaultUser(props: Omit<P, 'user'> & { userId?: number }) {
    const { userId, ...rest } = props
    // Inyecta el user completo a partir del userId
    const user = useUser(userId ?? 1)
    return <Component {...(rest as P)} user={user} />
  }
}
```

```
// Antes: el consumidor tiene que pasar el objeto user completo
<ProfileCard user={{ id: 1, name: 'María', avatar: '...' }} />
```



El caso de uso principal de un HOC es **manipular props antes de pasarlas**: inyectar datos, transformar formatos, agregar defaults o condicionar el render. Ejemplos conocidos: `React.memo` , `forwardRef` , `Redux connect()` .

Libreria utiles – Zustand

Estado global sin boilerplate

Estado Global: El Problema

React tiene `useState` para estado local y `useContext` para estado compartido. Pero Context tiene limitaciones.



Problemas de useContext

- × Re-renders masivos — cualquier cambio en el contexto re-renderiza TODOS los consumidores
- × Boilerplate — necesitas Provider, createContext, y un hook wrapper
- × No hay selectores — no puedes suscribirte a solo una parte del estado



Lo que necesitamos

- ✓ Estado accesible desde cualquier componente
- ✓ Re-renders solo cuando cambia lo que usas
- ✓ API simple, sin boilerplate
- ✓ TypeScript nativo

Zustand — Estado Global Simple

“ Definición

"Zustand es una librería de estado global minimalista. Un store es un hook — sin providers, sin boilerplate."

```
import { create } from 'zustand'

interface TodoStore {
  todos: Todo[]
  filter: 'all' | 'active' | 'done'
  addTodo: (text: string) => void
  toggleTodo: (id: number) => void
  setFilter: (filter: TodoStore['filter']) => void
}

const useTodoStore = create<TodoStore>((set) => ({
  todos: [],
  filter: 'all',
  addTodo: (text) => set((state) => ({
    todos: [ ...state.todos, { id: Date.now(), text, done: false } ],
  })),
  toggleTodo: (id) => set((state) => ({
```

Zustand — Uso en Componentes

El store es un hook — úsalo directamente en cualquier componente

```
function TodoList() {  
  // ✅ Solo se re-renderiza cuando 'todos' cambia  
  const todos = useTodoStore((state) => state.todos)  
  const toggleTodo = useTodoStore((state) => state.toggleTodo)  
  
  return todos.map(todo => (  
    <div key={todo.id} onClick={() => toggleTodo(todo.id)}>  
      {todo.text}  
    </div>  
  ))  
}  
  
function FilterBar() {  
  // ✅ Solo se re-renderiza cuando 'filter' cambia  
  const filter = useTodoStore((state) => state.filter)  
  const setFilter = useTodoStore((state) => state.setFilter)  
  
  return <select value={filter} onChange={e => setFilter(e.target.value)} />  
}
```



El **selector** `(state) => state.todos` es la clave: Zustand solo re-renderiza el componente cuando el valor seleccionado cambia. Sin selectores, sería como Context.

Zustand vs Context – Comparación

useContext

```
// 1. Crear contexto
const TodoContext = createContext<TodoStore>(null!)

// 2. Crear provider
function TodoProvider({ children }) {
  const [todos, setTodos] = useState([])
  const [filter, setFilter] = useState('all')
  const value = useMemo(() => ({
    todos, filter, setTodos, setFilter
  }), [todos, filter])

  return (
    <TodoContext.Provider value={value}>
      {children}
    </TodoContext.Provider>
  )
}

// 3. Hook wrapper
const useTodos = () => useContext(TodoContext)
```

Zustand

```
// ¡Eso es todo!
const useTodoStore = create<TodoStore>((set) => ({
  todos: [],
  filter: 'all',
  addTodo: (text) => set((state) => ({
    todos: [ ... state.todos, {
      id: Date.now(), text, done: false
    } ],
  })),
  setFilter: (filter) => set({ filter }),
}))

// Uso directo – sin provider
const todos = useTodoStore(s => s.todos)
```

Resumen



useEffect Cleanup Retorna una función para limpiar timers, listeners y suscripciones. Evita memory leaks.



memo + useCallback memo evita re-renders si props no cambian. useCallback estabiliza funciones. Se necesitan mutuamente.



Custom Hooks Extraen lógica reutilizable (useDebounce, useLocalStorage). Nombre siempre empieza con 'use'.



Composición y HOCs Children, slots y HOCs para UIs flexibles. Hoy se prefieren hooks, pero HOCs siguen siendo útiles.



Zustand Estado global sin boilerplate. Un store = un hook. Selectores para re-renders eficientes.

¿Preguntas?

Clase 9 – Advanced React



PONTIFICIA
UNIVERSIDAD
CATÓLICA
DE CHILE

IIC2182 – Primer Semestre 2026